- **Name:** Ahmad Mujtaba
- **Program:** Electrical Engineering (AI)
- **Semester:**2nd
- **Section:** B
- **Registration No:** 5125370 / BF25NWELE0706

Lab:5

The While Loop in C++Objectives

- To understand the concept of **loops in C++**.
- To learn how to use the **while loop**.
- To understand the concept of **nested loops**.
- To implement the **nested while loop**.
- To learn the working of the **do-while loop**.
- To compare **while loop and do-while loop**.

Theory

Loop

A **loop** is a programming structure that allows a set of instructions to be executed **repeatedly** until a certain condition becomes false.

Loops help programmers:

- reduce code repetition
- make programs efficient
- perform repetitive tasks easily.

Common loops in C++:

- while loop
- do-while loop
- for loop

While Loop

Definition

A **while loop** repeatedly executes a block of code **as long as the condition is true**.

The condition is checked **before the execution** of the loop body.

If the condition is **false at the beginning**, the loop will **not execute even once**.

Syntax

```
while(condition)
{
    // statements
}
```

Flow of Execution

1. Condition is checked.
2. If true → loop body executes.
3. Control goes back to condition.
4. If false → loop stops.

Example

```
#include <iostream>
using namespace std;

int main()
{
    int i = 1;

    while(i <= 5)
    {
        cout << i << endl;
        i++;
    }

    return 0;
}
```

Output

```
1
2
```

```
3
4
5
```

Nested While Loop

Definition

A **nested loop** means placing **one loop inside another loop**.

In nested loops:

- the **outer loop controls rows**
- the **inner loop controls columns**

Nested loops are commonly used for:

- patterns
- matrices
- tables

Syntax

```
while(condition1)
    while(condition2)
        // statements
```

Example

```
#include <iostream>
using namespace std;

int main()
{
    int i = 1;

    while(i <= 3)
    {
        int j = 1;

        while(j <= 3)
        {
            cout << "* ";
            j++;
        }

        cout << endl;
        i++;
    }

    return 0;
}
```

Output

```
* * *
* * *
* * *
```

---

5. Do-While Loop

Definition

A **do-while loop** is similar to a while loop, but the condition is checked **after the execution of the loop body**.

Therefore, the loop **executes at least one time** even if the condition is false.

Syntax

```
do
{
    // statements
}
while(condition);
```

⚠ A **semicolon (;)** is required after the condition.

Example

```
#include <iostream>
using namespace std;

int main()
{
    int i = 1;

    do
    {
        cout << i << endl;
        i++;
    }
    while(i <= 5);

    return 0;
}
```

Output:1 2 3 4 5

6. Difference Between While and Do-While Loop

Feature	While Loop	Do-While Loop
Condition check	Before loop	After loop
Minimum execution	0 times	At least 1 time
Syntax	while(condition)	do {} while(condition);

Lab 05 Task 1: While Loop

Write a C++ program that uses a **while loop** to display numbers from **1 to 10**, each on a **separate line**.

```
#include <iostream>

using namespace std;

int main(){

        int i = 1;

        while(i <= 10){

                cout << i << endl;

                 i++;

                }

            return 0;}
```

Output: 1 2 3 4 5 6 7 8 9 10

Task 2: Using do-while Loop

Repeat the **first task** using a **do-while loop**.

```
#include <iostream>
using namespace std;

int main()
{
    int i = 1;

    do
    {
        cout << i << endl;
        i++;
    }
    while(i <= 10);

    return 0;
}
```

Output: 1 2 3 4 5 6 7 8 9 10

Task 3: Sum Using While Loop

Write a C++ program that:

1. Prompts the user to enter a **positive integer**.
2. Uses a **while loop** to calculate the **sum from 1 to that number**.
3. Displays the calculated **sum**.

```
#include <iostream>
using namespace std;

int main(){
    int n, i = 1, sum = 0;

    cout << "Enter a positive integer: ";
    cin >> n;

    while(i <= n)
    {
        sum = sum + i;
        i++;
    }
    cout << "Sum = " << sum << endl;
    return 0;}
```

Example Output

```
Enter a positive integer: 5
Sum = 15
```